

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load an image (you can replace 'images.jpg' with a medical image if available)
# Using '/content/images.jpg' as it exists in the kernel state
image_path = '/content/images.jpg'
original_image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE) # Load as grayscale for simplicity

if original_image is None:
    print(f"Error: Could not load image from {image_path}. Please check the path.")
else:
    print(f"Original image dimensions: {original_image.shape[1]}x{original_image.shape[0]} pixels")
    print(f"Original image bit depth (estimated from data type): {original_image.dtype}")

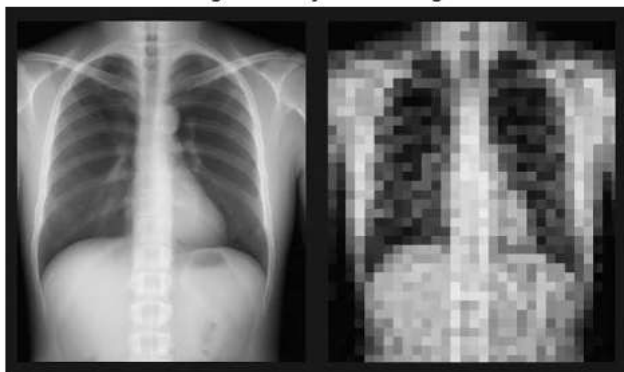
    plt.figure(figsize=(6, 6))
    plt.imshow(original_image, cmap='gray')
    plt.title('Original Grayscale Image')
    plt.axis('off')
    plt.show()

```

Original image dimensions: 718x426 pixels

Original image bit depth (estimated from data type): uint8

Original Grayscale Image



```

if original_image is not None:
    # Define target pixel resolutions
    low_res_1 = (original_image.shape[1] // 4, original_image.shape[0] // 4) # 1/4th resolution
    low_res_2 = (original_image.shape[1] // 8, original_image.shape[0] // 8) # 1/8th resolution

    # Downsample and then upsample to original size to observe detail loss
    img_low_res_1 = cv2.resize(original_image, low_res_1, interpolation=cv2.INTER_AREA)
    img_low_res_1_upscaled = cv2.resize(img_low_res_1, (original_image.shape[1], original_image.shape[0]), interpolation=cv2.INTER_AREA)

    img_low_res_2 = cv2.resize(original_image, low_res_2, interpolation=cv2.INTER_AREA)
    img_low_res_2_upscaled = cv2.resize(img_low_res_2, (original_image.shape[1], original_image.shape[0]), interpolation=cv2.INTER_AREA)

    plt.figure(figsize=(15, 5))

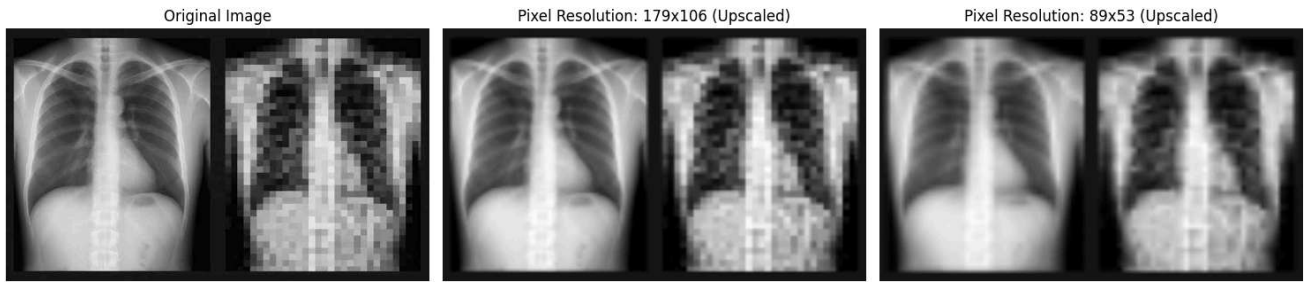
    plt.subplot(1, 3, 1)
    plt.imshow(original_image, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(img_low_res_1_upscaled, cmap='gray')
    plt.title(f'Pixel Resolution: {low_res_1[0]}x{low_res_1[1]} (Upscaled)')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(img_low_res_2_upscaled, cmap='gray')
    plt.title(f'Pixel Resolution: {low_res_2[0]}x{low_res_2[1]} (Upscaled)')
    plt.axis('off')

    plt.tight_layout()
    plt.show()

```



```
if original_image is not None:
    # Convert original image to float for accurate intensity level reduction
    img_float = original_image.astype(np.float32)

    # Normalize intensity to 0-1 range (assuming 8-bit original)
    normalized_img = img_float / 255.0

    # Reduce intensity levels
    # For 16 levels (4-bit effective):
    levels_16 = 16
    reduced_intensity_16 = np.round(normalized_img * (levels_16 - 1)) / (levels_16 - 1) * 255
    reduced_intensity_16 = reduced_intensity_16.astype(np.uint8)

    # For 4 levels (2-bit effective):
    levels_4 = 4
    reduced_intensity_4 = np.round(normalized_img * (levels_4 - 1)) / (levels_4 - 1) * 255
    reduced_intensity_4 = reduced_intensity_4.astype(np.uint8)

    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(original_image, cmap='gray')
    plt.title('Original Image (256 levels)')
    plt.axis('off')

    plt.subplot(1, 3, 2)
    plt.imshow(reduced_intensity_16, cmap='gray')
    plt.title(f'Intensity Resolution: {levels_16} levels (4-bit)')
    plt.axis('off')

    plt.subplot(1, 3, 3)
    plt.imshow(reduced_intensity_4, cmap='gray')
    plt.title(f'Intensity Resolution: {levels_4} levels (2-bit)')
    plt.axis('off')

    plt.tight_layout()
    plt.show()
```

